Burmese pays more than its neighbours

By **Aung Thu Hein** · 29 September 2026

AI models charge by the token. We counted how many tokens 10 tokenizers, used by GPT, Claude, Gemini, Llama, Qwen and others, need for the same 1,012 sentences in English, Burmese and six other Asian languages.

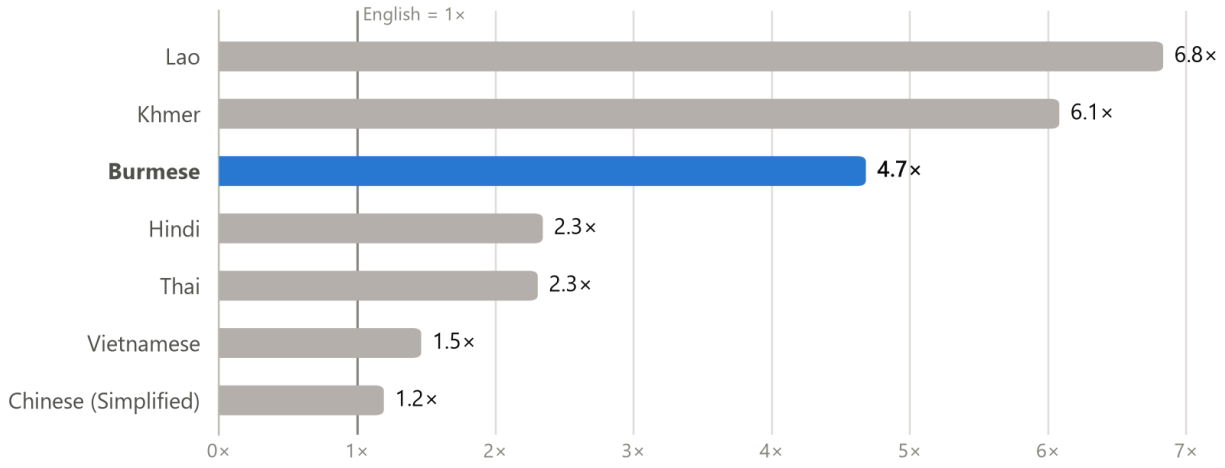
2.3×
Burmese costs this much more than Thai for the same meaning, on average (it's costlier with all 10 tokenizers)

5 of 10
tokenizers make Burmese the most expensive of all seven languages tested

11.7×
English, the worst case (Llama 3), close to one token per byte of text

Median token cost relative to English

Tokens for the same 1,012 FLORES-200 sentences ÷ tokens for English, median of 10 distinct tokenizers



Across the region, Burmese sits in the expensive group. With the typical tokenizer it needs **4.7×** the tokens of English, twice Thai (2.3×) and Hindi (2.3×), and far more than Vietnamese (1.5×) or Chinese (1.2×). It is costlier than Thai, Hindi, Vietnamese and Chinese with **every** tokenizer we tested.

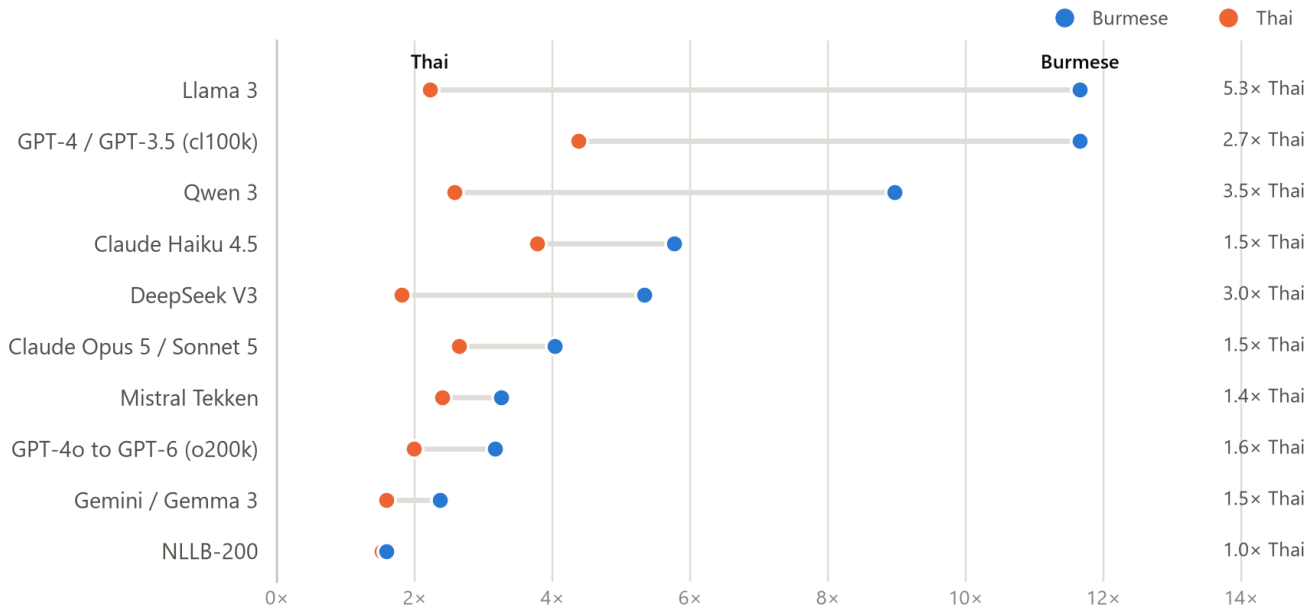
> **One honest caveat.** Burmese isn't the only language paying this tax. Lao (6.8×) and Khmer (6.1×) have even higher medians, and Burmese beats them with only 6 and 5 of 10 tokenizers respectively. The pattern is about *scripts with little training data*, and Burmese is one of the hardest hit.

Burmese vs Thai: same region, very different bills

Thai is the fairest comparison: a Southeast Asian language with its own script and no spaces between words, just like Burmese. Yet every tokenizer is kinder to Thai, and the gap is widest for older and English- or Chinese-centred vocabularies: with Llama 3, Burmese costs **5.3×** as much as Thai; with Qwen 3, **3.5×**. Only NLLB-200, a translation model built for 200 languages, treats the two almost equally.

Tokens relative to English, Burmese vs Thai

Each row is one tokenizer; the label on the right shows how many times more Burmese costs than Thai



All languages, all tokenizers

Tokens relative to English (1.0 = same as English). Darker = more expensive. Burmese column outlined.

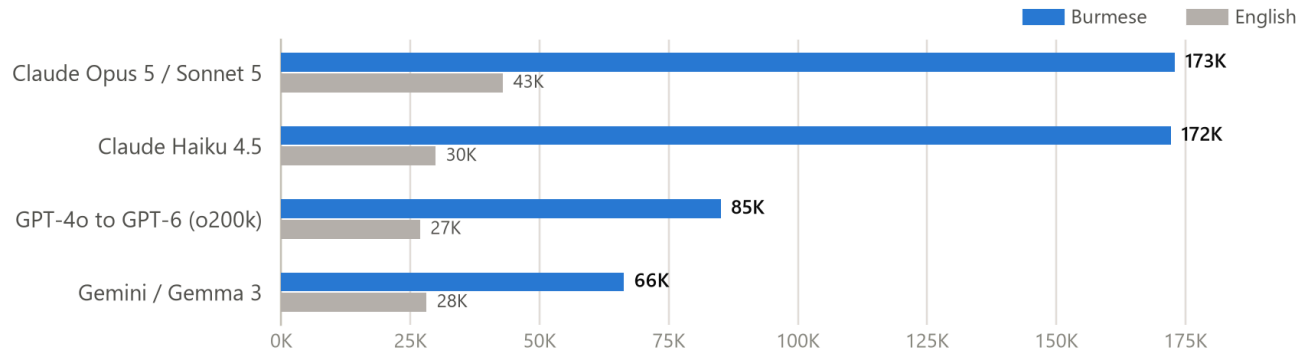
	Burmese	Thai	Khmer	Lao	Hindi	Vietnamese	Chinese
Llama 3	**11.7**	2.2	8.8	8.8	2.5	1.4	1.3
GPT-4 / GPT-3.5 (cl100k)	**11.7**	4.4	8.8	9.5	4.8	2.4	1.9
Qwen 3	**9.0**	2.6	6.5	5.7	4.4	1.4	1.0
Claude Haiku 4.5	**5.8**	3.8	5.7	5.7	3.1	2.4	1.6
DeepSeek V3	**5.3**	1.8	6.8	8.0	2.8	2.1	0.9
Claude Opus 5 / Sonnet 5	**4.0**	2.6	4.0	4.0	2.2	1.8	1.1
Mistral Tekken	**3.2**	2.4	14.7	12.7	1.8	1.4	1.4
GPT-4o to GPT-6 (o200k)	**3.2**	2.0	3.3	8.2	1.6	1.5	1.2
Gemini / Gemma 3	**2.4**	1.6	2.7	2.6	1.3	1.4	1.1
NLLB-200	**1.6**	1.5	1.8	1.5	1.2	1.2	1.1

What it means for the bill

The multiplier compares a tokenizer with itself. To compare **providers**, count the absolute tokens for the same text: that's what you pay for (times each provider's price per token).

Tokens for the same 1,012 sentences

Thousands of tokens, English vs Burmese, for the three major API providers' current tokenizers



Tokenizer	English tokens	Burmese tokens	Burmese ÷ English
Gemini / Gemma 3	28,125	**66,306**	2.4×
GPT-4o to GPT-6 (o200k)	26,878	**85,034**	3.2×
Claude Haiku 4.5	29,894	**172,181**	5.8×
Claude Opus 5 / Sonnet 5	42,927	**172,929**	4.0×

Gemini needs the fewest tokens for Burmese. Claude needs about **2.0×** as many as GPT and **2.6×** as many as Gemini. Claude's newer tokenizer (Opus 5 / Sonnet 5) shows a lower multiplier than Haiku 4.5 only because it uses more tokens for English; for Burmese the two are almost identical.

Why it happens

Tokenizers learn their vocabulary from training data that is mostly English, code and a few large languages. A script that is rare in that data gets few dedicated tokens, so its text is split into small fragments, down to raw UTF-8 bytes. Each Burmese character takes 3 bytes, so byte-level fallback is especially costly. The word မြန်မာ ("Myanmar"), 6 characters and 18 bytes, needs:

Tokenizer	GPT-4 (cl100k)	Qwen 3	GPT-4o to GPT-6	Gemini / Gemma 3	NLLB-200
Tokens for မြန်မာ	**12**	**9**	**3**	**3**	**1**

What Burmese developers can do

- **Pick the tokenizer, not just the model.** For Burmese-heavy workloads, Gemini / Gemma 3 needs 2.4× English against 11.7× for Llama 3. That gap is money, latency and context.
- **Budget context in Burmese tokens.** A 128K window holds roughly 408 Burmese sentences with Llama 3, against 4,766 in English.

- **Chunk by syllables, not characters,** when building retrieval systems, so chunks stay even in meaning (see *Burmese RAG*).
- **Measure your own text.** Run `mtb run --corpus your_folder/` on your real documents.

Method and limitations

Data: FLORES-200 devtest (1,012 sentences from Wikinews, Wikijunior and Wikivoyage, professionally translated; CC BY-SA 4.0). Each language's sentences are joined by newlines and tokenized without special tokens. The multiplier is tokens(language) ÷ tokens(English) for the same tokenizer. Models whose counts are identical in every language share a tokenizer and are counted once (10 distinct tokenizers): GPT-5, GPT-5.5, GPT-6 Sol and Phi-4 mini match o200k; Gemini 2.5 and 3.8 Flash match Gemma 3; Claude Sonnet 5 matches Opus 5. Claude, GPT-5+ and Gemini were counted with their official token-counting APIs, with the fixed message framing subtracted; Llama 3, Gemma 3 and Mistral tokenizers come from public mirrors of the official files. Limitations: news and wiki text only; Unicode Burmese only (Zawgyi not measured); token efficiency is not answer quality; provider tokenizers can change with new models. Run date: 2026-09-29.

Code and full data: **github.com/aungthuhein2005/myanmar-tokenizer-bench** (MIT).